

CANable 2.0 → Google Pixel 10

Developer Integration Specification

AiM MXP CAN Output | 2003 BMW M3 (E46) | AI Driving Coach

Document Version 1.0

1. System Overview

The AiM MXP dash logger transmits vehicle and sensor data over its CAN2 output port to the Jhoinrch RH-02 PRO (CANable 2.0) adapter. The adapter converts the CAN bus to a USB serial stream using the SLCAN protocol, delivered to the Google Pixel 10 coaching engine via USB-C OTG.

Component	Detail
Vehicle	2003 BMW M3 (E46)
Dash Logger	AiM MXP
CAN Bus (MXP CAN2 Output)	1 Mbit/s 11-bit standard frames Little Endian
CAN-to-USB Adapter	Jhoinrch RH-02 PRO (CANable 2.0) — SLCAN firmware
USB Connection	USB-C to Pixel 10 (OTG)
Receive Target	Google Pixel 10 (AI Coaching Engine)
Total CAN Frames	8 frames (0x420-0x424, 0x450-0x452)
Total Channels	29 channels

2. Physical Connection

2.1 Wiring

Connect the RLCAB015L cable from the MXP CAN2 port to the RH-02 PRO screw terminal as follows:

RLCAB015L Wire	RH-02 PRO Terminal	Notes
Blue	CAN H	
White	CAN L	
GND	Leave unconnected	RH-02 PRO is galvanically isolated — GND not required
5V	Leave unconnected	5V terminal is an output, not required for operation

2.2 Termination

The MXP has its internal 120 ohm termination resistor enabled on CAN2. Enable the termination switch on the RH-02 PRO as well. Two nodes, one resistor at each end — correct bus topology.

2.3 Android USB-C OTG

Connect the RH-02 PRO USB-C output to the Pixel 10 USB-C port using a USB-C OTG cable or direct USB-C to USB-C connection. The Pixel 10 must request USB host mode permissions for the CDC serial device on first connection.

3. SLCAN Protocol

3.1 Transport

The RH-02 PRO exposes the CAN bus as a USB CDC (Communications Device Class) virtual serial device. On Android it appears as a USB serial device compatible with the usb-serial-for-android library. Each CAN frame is delivered as a terminated ASCII string.

3.2 Frame Format

Every incoming frame follows this fixed format:

```
t<ID><DLC><DATA>\r
```

Field	Example (0x420)	Description
Frame type	t	Lowercase 't' = standard 11-bit frame
CAN ID	420	3 hex digits, no 0x prefix
DLC	8	Data length in bytes (always 8 in this protocol)
Data	XXXXXXXXXXXXXXXXXX	16 hex characters = 8 bytes of payload
Terminator	\r	Carriage return (0x0D) ends every frame

Example — Frame 0x420 with 8 bytes of data:

```
t42081A2B3C4D5E6F7A8B\r
```

NOTE: All frames in this protocol use 11-bit standard addressing (lowercase 't'). 29-bit extended frames would use uppercase 'T' but are not present in this protocol.

4. CAN Frame Summary

Eight frames are broadcast continuously by the AiM MXP on CAN2 at 1 Mbit/s. All frames are 8 bytes, 11-bit standard IDs, Little Endian byte order.

CAN ID	DLC	Rate	Contents
0x420	8B	10 Hz	RPM, GPS Speed, Gear, Water Temp
0x421	8B	50 Hz	Water Pressure, Roll Rate, Oil Filter Temp, Oil Pressure
0x422	8B	50 Hz	Brake Pressure, Pedal Position, Brake Switch, Pitch Rate
0x423	8B	50 Hz	Steering Angle, Yaw Rate, Lateral Accel, Inline Accel
0x424	8B	50 Hz	Fuel Level, Battery Voltage, Vertical Accel
0x450	8B	10 Hz	Wheel Speed FL, FR, RL, RR
0x451	8B	10 Hz	ECU Speed, Temp Outside, Engine Oil Temp, DSC REG
0x452	8B	20 Hz	GPS Latitude, GPS Longitude

WARNING: Frame frequency is set at the CAN ID level, not per channel. Frames 0x421-0x424 run at 50 Hz driven by the IMU channels they contain. Slower channels within those frames (e.g. Oil Filter Temp at 1 Hz source) will transmit repeated values between natural source updates. This is expected and harmless.

5. Channel Decode Reference

All multi-byte values are Little Endian. Byte 0 is the least significant byte. Apply the decode formula to the raw integer extracted from each channel's byte range to obtain the engineering value.

CAN ID	Bytes	Channel	Format	Decode Formula	Output Unit	Notes
0x420	0-1	RPM	Unsigned	value x 1	rpm	
0x420	2-3	GPS Speed	Unsigned	value x 0.1	mph	Primary speed source
0x420	4-5	Gear	Unsigned	value x 1	gear	Not available on MSS54HP — will read 0
0x420	6-7	Water Temp	Unsigned	value x 0.1	deg F	
0x421	0-1	Water Pressure	Unsigned	value x 0.1	psi	Analog sensor
0x421	2-3	Roll Rate	Signed *	value x 0.1	deg/s	Verify sign on first capture
0x421	4-5	Oil Filter Temp	Unsigned	value x 0.1	deg F	Primary oil health indicator
0x421	6-7	Oil Pressure	Unsigned	value x 0.1	psi	Analog sensor
0x422	0-1	Brake Pressure	Unsigned	value x 1	psi	Analog sensor
0x422	2-3	Pedal Position	Unsigned	value x 0.01	%	0-100%
0x422	4-5	Brake Switch	Unsigned	0=released 1=applied	binary	

CAN ID	Bytes	Channel	Format	Decode Formula	Output Unit	Notes
0x422	6-7	Pitch Rate	Signed *	value x 0.1	deg/s	Verify sign on first capture
0x423	0-1	Steering Angle	Signed *	value x 0.1	deg	Verify sign on first capture
0x423	2-3	Yaw Rate	Signed *	value x 0.1	deg/s	Verify sign on first capture
0x423	4-5	Lateral Accel	Signed *	value x 0.01	g	Verify sign on first capture
0x423	6-7	Inline Accel	Signed *	value x 0.01	g	Verify sign on first capture
0x424	0-1	Fuel Level	Unsigned	value x 0.01	gal	
0x424	2-3	Battery Voltage	Unsigned	value x 0.1	V	
0x424	4-5	Vertical Accel	Signed *	value x 0.01	g	Verify sign on first capture
0x450	0-1	Wheel Speed FL	Unsigned	value x 0.1	mph	From BMW DSC
0x450	2-3	Wheel Speed FR	Unsigned	value x 0.1	mph	From BMW DSC
0x450	4-5	Wheel Speed RL	Unsigned	value x 0.1	mph	From BMW DSC
0x450	6-7	Wheel Speed RR	Unsigned	value x 0.1	mph	From BMW DSC
0x451	0-1	ECU Speed	Unsigned	value x 0.1	mph	Cross-reference for GPS
0x451	2-3	Temp Outside	Unsigned	value x 0.1	deg F	Ambient temperature
0x451	4-5	Engine Oil Temp	Unsigned	value x 0.1	deg F	DME sump sensor — secondary ref
0x451	6-7	DSC REG	Unsigned	0=inactive 1=intervening	binary	
0x452	0-3	GPS Latitude	Signed (4B)	value x 0.0000001	decimal deg	deg7 format, +N hemisphere
0x452	4-7	GPS Longitude	Signed (4B)	value x 0.0000001	decimal deg	Negative = W hemisphere

NOTE: Channels marked Signed * are bidirectional. See Section 6 for sign convention details and the recovery formula for SLCAN unsigned representation of negative values.

6. Signed Channel Handling

6.1 Sign Recovery

SLCAN transmits all data as unsigned hex bytes. For signed 2-byte channels, the receiver must cast the raw 16-bit value to a signed short before applying the decode multiplier. In Java/Android:

```
short rawSigned = (short)((data[highByte] & 0xFF) << 8 | (data[lowByte] & 0xFF));
double value = rawSigned * multiplier;
```

If raw value > 32767 and a signed short cast is not available, use:

```
int raw = ...; // unsigned read
if (raw > 32767) raw = raw - 65536; // recover signed value
double value = raw * multiplier;
```

6.2 Sign Convention — Verify on First Capture

The following channels are bidirectional. Verify sign convention on first engine-running data capture. If a channel produces large positive values where negatives are expected, negate the decoded value in the decoder.

Channel	Positive (+)	Negative (-)	Action if wrong
Roll Rate	Roll right	Roll left	Negate value in decoder
Pitch Rate	Nose up	Nose down	Negate value in decoder
Yaw Rate	Rotate right	Rotate left	Negate value in decoder
Lateral Accel	Cornering right	Cornering left	Negate value in decoder
Inline Accel	Acceleration	Braking	Negate value in decoder
Vertical Accel	Bump (upward)	Rebound (downward)	Negate value in decoder
Steering Angle	Right of centre	Left of centre	Negate value in decoder

7. GPS Coordinate Decoding

Frame 0x452 carries GPS Latitude (bytes 0-3) and GPS Longitude (bytes 4-7) as signed 4-byte integers in deg7 format (decimal degrees x 10,000,000). This provides approximately 1.1 cm resolution globally.

7.1 Decode

```
int rawLat = (data[3]&0xFF)<<24 | (data[2]&0xFF)<<16 | (data[1]&0xFF)<<8 |
(data[0]&0xFF);
int rawLon = (data[7]&0xFF)<<24 | (data[6]&0xFF)<<16 | (data[5]&0xFF)<<8 |
(data[4]&0xFF);
double lat = rawLat * 0.0000001; // e.g. +37.xxxxxxx (Northern California)
double lon = rawLon * 0.0000001; // e.g. -121.xxxxxx (negative = West hemisphere)
```

NOTE: Northern California produces positive latitude (~+37 deg) and negative longitude (~-122 deg). Ensure the 4-byte signed integer decode is used — do not use unsigned decode for this frame.

8. Android Integration

8.1 Recommended Library

Use the `usb-serial-for-android` library (github.com/mik3y/usb-serial-for-android) to communicate with the RH-02 PRO CDC serial device on the Pixel 10.

8.2 Sample Decode Code

The following Java/Android snippet demonstrates opening the serial port, reading SLCAN frames, and decoding key channels:

```
// 1. Add dependency to build.gradle:
//      implementation 'com.github.mik3y:usb-serial-for-android:3.4.6'

// 2. Open the SLCAN serial port
UsbManager usbManager = (UsbManager) getSystemService(Context.USB_SERVICE);
UsbSerialDriver driver = UsbSerialProber.getDefaultProber()
    .probeDevice(usbManager.getDeviceList().values().iterator().next());
UsbSerialPort port = driver.getPorts().get(0);
port.open(usbManager.openDevice(driver.getDevice()));
port.setParameters(115200, 8, UsbSerialPort.STOPBITS_1,
    UsbSerialPort.PARITY_NONE);

// 3. Read and parse SLCAN frames
byte[] buffer = new byte[256];
int len = port.read(buffer, 1000);
String frame = new String(buffer, 0, len).trim(); // e.g.
"t4208XXXXXXXXXXXXXXXXXX"

// 4. Parse frame
if (frame.startsWith("t")) {
    int canId = Integer.parseInt(frame.substring(1, 4), 16); // e.g. 0x420
    int dlc    = Integer.parseInt(frame.substring(4, 5));    // always 8
    byte[] data = hexToBytes(frame.substring(5, 5 + dlc * 2));
}

// 5. Decode 0x420 example (Little Endian, unsigned 2-byte integers)
int rpm      = (data[1] & 0xFF) << 8 | (data[0] & 0xFF); // bytes 0-1
int rawSpeed = (data[3] & 0xFF) << 8 | (data[2] & 0xFF); // bytes 2-3
int gear     = (data[5] & 0xFF) << 8 | (data[4] & 0xFF); // bytes 4-5
int rawWTemp = (data[7] & 0xFF) << 8 | (data[6] & 0xFF); // bytes 6-7
double speed  = rawSpeed * 0.1; // mph
double wTemp  = rawWTemp * 0.1; // deg F

// 6. Decode signed channel example (Lateral Accel from 0x423 bytes 4-5)
short rawLatG = (short)((data[5] & 0xFF) << 8 | (data[4] & 0xFF));
double latG    = rawLatG * 0.01; // g (sign preserved by short cast)

// 7. Decode GPS Latitude from 0x452 bytes 0-3 (4-byte signed Little Endian)
int rawLat = (data[3] & 0xFF) << 24 | (data[2] & 0xFF) << 16
```

```

        | (data[1] & 0xFF) << 8 | (data[0] & 0xFF);
double lat = rawLat * 0.0000001; // decimal degrees

```

8.3 USB Permissions

Add to AndroidManifest.xml:

```

<uses-feature android:name="android.hardware.usb.host" />

```

Request USB permission at runtime before opening the port:

```

PendingIntent permIntent = PendingIntent.getBroadcast(this, 0,
    new Intent(ACTION_USB_PERMISSION), 0);
usbManager.requestPermission(device, permIntent);

```

9. Known Constraints and Outstanding Items

Item	Constraint
Gear Position (0x420 bytes 4-5)	Not available on BMW MSS54HP DME on any path. Channel will always read 0. Do not use for coaching logic.
Battery Voltage (0x424 bytes 2-3)	Scale confirmed at value x 0.1 (V). Verify range on first capture.
Signed channels via SLCAN	If value > 32767 on a signed channel, apply: actual = value - 65536 to recover the true negative value.
Oil Filter Temp (0x421 bytes 4-5)	PT-100 analog sensor at filter housing. Primary oil health indicator. Slow thermal response — do not use for rapid event detection.
Engine Oil Temp (0x451 bytes 4-5)	BMW DME sump sensor. Lags significantly behind actual oil temp. Secondary reference only.
GPS Longitude (0x452 bytes 4-7)	Negative value for West hemisphere (Northern California = approx -122 deg). Ensure signed 4-byte decode.
Frame frequency mismatch	Frames 0x421-0x424 run at 50 Hz (driven by IMU channels). Slower channels in those frames (oil temp, fuel) will repeat values between natural update intervals. This is expected and harmless.

10. CAN ID Collision Registry

Verified: Zero collisions across all three buses on the AiM MXP.

CAN ID	Bus	Owner	Purpose
0x420-0x424	CAN2 Output	AiM MXP	Standard channels — RPM, speed, IMU, temps, pressures
0x450-0x452	CAN2 Output	AiM MXP	Extended channels — wheel speeds, ECU supplemental, GPS

CAN ID	Bus	Owner	Purpose
BMW DME/DS C IDs	CAN1 (receive)	BMW DME	Engine/chassis data received only, not retransmitted
0x18FEF433	CAN2 (receive)	TPMS	Receive only — isolated, not retransmitted on CAN Output
0x420-0x456	SmartyCam port	AiM MXP	Independent physical bus to VBOX HD2 — no collision possible